STM32 Lernskript – Stack, Assembler & Buffer Overflow

Dieses Dokument fasst strukturiert und verständlich die wichtigsten Inhalte aus dem Chat zusammen. Ziel ist es, das Debugging auf STM32 (Cortex-M), das Lesen von Assembler-Code und das Verständnis von Stack-basierten Buffer Overflows sicher zu beherrschen.

1. STM32 & Cortex-M Grundlagen

STM32-Mikrocontroller (hier: STM32F302, Cortex-M4) verwenden den ARM Thumb-2 Instruktionssatz. Programme laufen im Flash (ab 0x08000000), der Stack liegt im RAM (meist ab 0x20000000). Der Stack wächst zu kleineren Adressen.

2. ARM Calling Convention (AAPCS)

Die ARM Procedure Call Standard (AAPCS) definiert eindeutig, wie Funktionen aufgerufen werden: - r0–r3: Funktionsargumente 1–4 - r0: Rückgabewert - lr (r14): Rücksprungadresse - sp (r13): Stack Pointer - r7: Frame Pointer (bei -O0 häufig genutzt)

3. Wichtige Assembler-Instruktionen

mov / mov.w: Kopiert Register oder Immediate-Werte.
bl: Branch with Link – Funktionsaufruf, speichert Rücksprungadresse in lr.
bx lr: Rücksprung aus Funktion.
push {r7, lr}: Speichert Register auf dem Stack.
pop {r7, pc}: Stellt Register wieder her und springt zurück.

4. Stack-Frame Aufbau (Beispiel vulnerable())

Beim Funktionsaufruf wird ein Stack-Frame angelegt. Beispiel:

```
push {r7, lr}
sub sp, #24
add r7, sp, #0
```

Bedeutung: - r7 zeigt auf den Beginn der lokalen Variablen - 24 Bytes Stack-Speicher werden reserviert - Lokale Variablen und gesicherte Register liegen fest relativ zu r7

5. Buffer Overflow Mechanismus

Ein Buffer Overflow entsteht nicht durch das Ablegen eines Wertes auf dem Stack, sondern durch das Kopieren von mehr Daten als der Zielpuffer aufnehmen kann. Funktionen wie strcpy prüfen keine Längen und schreiben Byte für Byte weiter.
Wenn über das Ende eines lokalen Puffers hinaus geschrieben wird, werden gespeicherte Register (z. B. r7 und lr) überschrieben. Beim pop {r7, pc} wird dann eine manipulierte Rücksprungadresse geladen.

6. Typisches Overflow-Symptom

- Stack enthält Muster wie 0x41414141 ('AAAA') - lr oder pc zeigen auf ungültige Adresse - CPU springt ins Nirvana → HardFault

7. Debugging-Werkzeuge

- STM32CubeIDE Debug View (Stack Frames) - Registers View (sp, lr, pc, r7) - Disassembly View - Memory View (Stack bytegenau ansehen) - arm-none-eabi-objdump für vollständige Disassembly

8. Lernziel & Sicherheit

Ziel dieses Lernskripts ist es, Embedded-Software robuster zu entwickeln. Das Verständnis von Overflows erklärt, warum sichere Funktionen, MPU, Stack-Checks und saubere API-Nutzung wichtig sind. Die Inhalte dienen ausschließlich Lern- und Debugging-Zwecken.

Ende des Lernskripts Plattform: STM32 Nucleo (STM32F302, Cortex-M4) Werkzeuge: STM32CubeIDE, GCC, GDB